

ArtGrid: Computer Vision-Assisted Spatial Mapping Mobile Application for Digital-to-Physical Proportional Scaling

A report submitted for the course named Project - I (CS3201)

Submitted By

YOGITA KUMARI
SEMESTER - VI
230101091

Supervised By

DR. R. K. BIDYALAKSHMI DEVI



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY (IIIT) SENAPATI, MANIPUR
APRIL, 2026

Declaration

In this submission, I have expressed my idea in my own words and I have adequately cited and referenced any ideas or words that were taken from another source. I also declare that I adhere to all principles of academic honesty and integrity and that I have not misrepresented or falsified any ideas, data, facts or sources in this submission. If any violation of the above is made, I understand that the institute may take disciplinary action. Such a violation may also engender disciplinary action from the sources which were not properly cited or permission not taken when needed.

YOGITA KUMARI
230101091

DATE: 30-04-2026



Department of Computer Science & Engineering
Indian Institute of Information Technology Senapati, Manipur

Dr. R. K. Bidyalakshmi Devi
Assistant Professor

Email: bidyalakshmi@iiitmanipur.ac.in
Contact No: +91 70051 26046

To Whom It May Concern

This is to certify that the project report entitled “**ArtGrid: Computer Vision-Assisted Spatial Mapping Mobile Application for Digital-to-Physical Proportional Scaling**”, submitted to the department of Computer Science and Engineering, Indian Institute of Information Technology (IIIT) Senapati, Manipur in partial fulfillment for the award of degree of Bachelor of Technology in Computer Science and Engineering is record bonafide work carried out by **Yogita Kumari** bearing roll number 230101091.

Signature of Supervisor

(Dr. R. K. Bidyalakshmi Devi)

Signature of Examiner 1

Signature of Examiner 2

Signature of Examiner 3

Signature of Examiner 4

Abstract

ArtGrid is a mobile application designed to replace the traditional static drawing grid. It acts as a digital proportional divider: given a reference photograph and a target canvas size, it maps pixel coordinates to real-world centimetre positions and overlays a calibrated metric grid on the reference image on screen, allowing an artist to read off exact measurements without manual calculation.

Beyond grid overlays, ArtGrid integrates a cloud machine learning pipeline capable of detecting human and animated faces, identifying objects, and extracting subjects from backgrounds. An on-device C++ image-processing library provides perceptual colour sampling in the Oklab colour space, a Kubelka–Munk paint-mix solver, edge extraction, Kuwahara filtering, perspective correction and palette quantisation; all without network access.

The backend consists of three independent microservices: an authentication service storing user identities in PostgreSQL with AES-256-GCM field-level encryption, a machine learning service running inference on a cloud GPU instance using ONNX Runtime and the dlib face landmark library, and an AI proxy that streams Gemini model responses token-by-token using Server-Sent Events so that API keys never reside on the device.

The primary contribution is the **Unified Human and Animated Face Detection** pipeline. A YOLOv8x6 model was trained on annotated Pixiv illustrations and integrated alongside a classical dlib HOG detector. A novel fusion layer named ArtGridFaceDomainFusion applies cross-head non-maximum suppression to produce a single ranked list of detections from both photorealistic and anime-style references, enabling the same centimetre measurement workflow across both artistic domains.

Acknowledgement

I express my profound gratitude to **Dr. R. K. Bidyalakshmi Devi**, assistant professor and a member of our esteemed faculty, for her unwavering guidance, continuous supervision and invaluable provision of project-related information. Her support played a pivotal role in the successful completion of this project. While I have invested significant effort in this endeavour, it is crucial to acknowledge the collective contribution of the faculty members within the Computer Science and Engineering Department at IIIT Manipur. Their keen interest in my project and consistent guidance were instrumental in bringing this project to fruition.

I would like to extend a special note of thanks to the Head of the Department (HoD) of Computer Science and Engineering, **Dr. Navanath Saharia**, for fostering a conducive environment for learning and research and for the continuous encouragement and leadership that has guided me through this project.

I extend my heartfelt thanks to all faculty members for their encouragement, which played a crucial role in making this project a success.

I also extend my deepest appreciation to my parents for providing me with all the necessary means and encouraging me in my pursuits of both education and life. Their unwavering support, whether by fulfilling my needs or nurturing my ambitions, has been invaluable in getting me this far. I would like to thank them for always believing in me and being my strongest support.

Yogita Kumari

Contents

1 Introduction	6
1.1 Prologue	6
1.2 Motivation	6
1.3 Technical Architecture	6
1.3.1 Android Client	7
1.3.2 Backend Microservices	7
1.3.3 On-Device versus Server Processing	7
1.4 Key Features and Novelty	7
1.5 Applications and Significance	8
2 Literature Survey	9
2.1 Computer Vision for Artistic Reference Analysis	9
2.2 Foundational Technologies	9
2.2.1 Object and Face Detection: The YOLO Family	9
2.2.2 Facial Landmark Detection: dlib and Ensemble Regression Trees	10
2.2.3 Image Segmentation: U ² -Net	10
2.2.4 Perceptual Colour Spaces: Oklab	10
2.2.5 Kubelka–Munk Paint Mixing Theory	10
2.2.6 Edge Detection, Kuwahara Filtering, and Perspective Correction	10
2.3 Animated Face Detection Research	11
2.3.1 Challenges of Detecting Anime-Style Faces	11
2.3.2 YOLOv8x6 Anime-Face Model	11
2.4 Large Language Models and SSE Streaming	12
2.5 Mobile Architecture and Microservices	12
2.6 Related Work and Positioning	13
2.7 Project Timeline	13
3 Requirement Engineering	14
3.1 Functional Requirements	14
3.2 Non-Functional Requirements	16
3.3 System Requirements	17
3.3.1 Software Requirements	17

3.3.2	Hardware Requirements	18
3.4	Use Cases	18
3.4.1	Use Case Diagram	18
3.4.2	Use Case Descriptions	18
4	System Design	20
4.1	System Architecture	20
4.2	Module Design	20
4.2.1	Android Client	20
4.2.2	artgrid-auth-service	22
4.2.3	artgrid-ml-service	22
4.2.4	artgrid-ai-proxy	23
4.2.5	Native C++ Image-Processing Library	23
4.3	Storage Design	25
4.3.1	PostgreSQL Schema	25
4.3.2	Room Database Schema	25
4.4	Data Flow Diagrams	26
4.4.1	Level 0 DFD	26
4.4.2	Level 1 DFD	26
4.5	Process Flowchart	27
4.6	Sequence Diagram	28
4.7	Component Diagram	29
4.8	State Machine Diagram	29
5	Implementation	31
5.1	Architecture Overview	31
5.2	ArtGridFaceDomainFusion: Unified Human and Animated Face Detection	31
5.2.1	Design Rationale	31
5.2.2	Parallel Inference Architecture	32
5.2.3	API Endpoint Implementation	32
5.2.4	Cross-Head NMS Fusion	32
5.3	C++ Native Image-Processing Library	33
5.3.1	JNI Bridge	33
5.3.2	Oklab Greyscale (C++ Implementation)	33
5.3.3	K-M Paint-Mix Solver	34
5.4	Android Application Implementation	34
5.4.1	Navigation Graph and Route Table	34
5.4.2	SSE Chat Stream Implementation	34

5.4.3	AuthInterceptor and Token Lifecycle	34
5.5	Split Authentication Design	35
5.6	GPU Deployment	35
6	Results	36
6.1	Test Environment and Pre-Requisites	36
6.2	Machine Learning Pipeline Results	36
6.2.1	Unified Human and Animated Face Detection	36
6.2.2	Object Detection	38
6.2.3	Background Segmentation	38
6.3	On-Device C++ Filter Performance	38
6.4	Testing and Validation	39
6.4.1	Functional verification	39
6.4.2	Integration paths	40
6.4.3	Performance and stress	40
6.4.4	Failure and edge cases	40
6.5	User Interface and Feature Results	41
7	Conclusion	44
7.1	Brief Rundown	44
7.2	Key Contributions	44
7.2.1	ArtGridFaceDomainFusion	44
7.2.2	Three-Microservice Backend with Dual Authentication	45
7.2.3	C++ Native Image-Processing Library	45
7.3	Limitations	45
7.4	Future Directions	46
7.5	Final Remarks	46
	Appendix	47
	Bibliography	51

List of Figures

2.1	Gantt Chart (19 January 2026 – 20 April 2026)	13
-----	---	----

3.1	Use Case Diagram	18
4.1	System Architecture	21
4.2	Data Flow Diagram Level 0	26
4.3	Data Flow Diagram Level 1	26
4.4	Flowchart	27
4.5	Sequence Diagram	28
4.7	State Machine Diagram	29
4.6	Component Diagram	30
6.1	UI mockups — primary navigation, offline tools, feature hub and canvas calibration.	41
6.2	UI mockups — colour sampling, physical paint mix, dominant- colour extraction, and shadow geometry.	42
6.3	UI mockups — face analysis, trace underlay, metric edge pre- view, and native filter selection.	43

List of Tables

1.1	On-Device vs. Server Processing Split in ArtGrid	7
1.2	Contributions vs. Standard Approaches	8
2.1	Anime-Face Detector Performance on Pixiv Test Set	12
2.2	Existing Systems vs. ArtGrid Contributions	13
3.3	Software Requirements for ArtGrid	17
3.4	Hardware Requirements for ArtGrid	18
4.1	PostgreSQL users Table (artgrid-auth-service)	25
4.2	Room Database Tables (ArtGrid Android)	25
5.1	ArtGrid Navigation Routes	34
5.2	Split Authentication Model	35
5.3	GPU Deployment Options for artgrid-ml-service	35
6.1	Test Environment	36
6.2	Unified Face Detection — Per-Head and Fused Results	37
6.3	YOLOv8x6 vs. YOLOv5-anime detection comparison	37
6.4	Inference Latency per Endpoint	37
6.5	On-Device C++ Filter Execution Times (Redmi 12 5G, 2048×1536 px)	39

Chapter 1

Introduction

1.1 Prologue

Translating proportions from a reference photograph onto a different-sized canvas usually means overlaying grids and redoing ruler arithmetic whenever the canvas or print size changes—tedious and easy to distort.

ArtGrid is an Android application that replaces that workflow. Given a reference image and target paper dimensions (A4/A3/custom cm), it maps pixels to physical centimetres and draws a calibrated grid on the reference on screen for direct read-off.

Beyond the grid, a cloud ML pipeline detects human and anime-style faces, objects, and foreground regions; results return to the device and align with the same cm grid for measurement-aware overlays.

1.2 Motivation

Ratio transfer across scales is a known hurdle for learners [1]. Typical tools are desktop-centric or passive viewing aids.

Two gaps motivate ArtGrid: **(1)** Anime and illustrated faces are underserved by detectors tuned only on photographs; unified human-and-anime inference addresses that. **(2)** Mobile colour widgets often expose only sRGB/HSL; painters need perceptual, Kelvin, and mix-oriented readouts supplied here by on-device native code.

1.3 Technical Architecture

Four parts: one Android client and three deployable backends (scale/update/security independently).

1.3.1 Android Client

Kotlin, Jetpack Compose, Hilt, Room [2]; Retrofit/Moshi for auth and ML APIs; a separate OkHttp client [3] for Gemini SSE because Retrofit buffers full bodies. A JNI-backed C++/NDK [4] library implements filters and colour science offline.

1.3.2 Backend Microservices

- **artgrid-auth-service** (Go/Chi [5], PostgreSQL): Google Sign-In → JWT; google_sub and email encrypted with AES-256-GCM (pgcrypto [6]).
- **artgrid-ml-service** (FastAPI, ONNX Runtime [7], dlib [8]): Face landmarks, unified face detection, YOLOv8n objects [9], U²-Net segmentation [10]; GPU VM (GCP [11] or RunPod [12]) for YOLOv8x6.
- **artgrid-ai-proxy**: SSE [13] Gemini tokens; hourly HMAC X-Device-Token; API key server-side only.

1.3.3 On-Device versus Server Processing

GPU-bound inference versus offline JNI work:

Table 1.1: On-Device vs. Server Processing Split in ArtGrid

Location	Capabilities
Server (GPU VM)	Human face detection (dlib HOG + face landmarks), Unified animated face detection (YOLOv8x6 ONNX), ArtGridFaceDomainFusion cross-head NMS, Object detection (YOLOv8n ONNX), Background segmentation (U ² -Net), Gemini AI chat (SSE stream)
On Device (JNI)	Edge extraction with λ sensitivity, Perspective correction (homography), Oklab perceptual greyscale, Tonal heatmap, Grey-world white balance, Linear invert, Kuwahara edge-preserving filter, Colour sampling (Oklab / HSL / Kelvin), K-M paint-mix solver, Dominant palette quantisation, Cm-grid and A-B ruler overlays

1.4 Key Features and Novelty

The original contributions of ArtGrid are distinguished from the use of publicly available models and libraries in Table 1.2.

Table 1.2: Contributions vs. Standard Approaches

Area	Standard Approach	ArtGrid Contribution
Face Analysis	Single domain detector (human only)	ArtGridFaceDomainFusion: parallel dlib + YOLOv8 ONNX heads, cross-head NMS, domain routing
Colour Tools	sRGB / HSL pickers	Oklab perceptual space, Kelvin temperature, K–M paint-mix solver, palette quantisation — all in native C++
Structural Filters	Third-party filter SDKs	Artist-specific chain: edges, tonal heatmap, perspective, Kuwahara, white balance, tied to cm-grid measurement UX
AI Chat	API key in mobile app	Dedicated Go SSE proxy; HMAC hourly device token; Gemini key server-side only
Identity Storage	Plaintext emails in DB	Self-hosted Go auth with AES-256-GCM encrypted PII in PostgreSQL
GPU Deployment	“Works on laptop”	Documented Docker Compose, NVIDIA Container Toolkit, GCP and RunPod GPU paths
ML Service Resilience	Assume service always online	Health-gated ML UI; distinct <i>Loading</i> , <i>Success</i> , <i>Empty</i> , and <i>Error</i> states

The primary novelty is ArtGridFaceDomainFusion: photoreal landmarks plus anime detection with cross-head NMS on one calibrated measurement surface.

1.5 Applications and Significance

Users include learners needing quantitative spatial feedback, anime or character artists underserved by generic detectors, and anyone using Oklab/Kelvin readouts with K–M-grounded mixing [14]. The codebase also illustrates JWT+HMAC patterns, SSE proxying, JNI colour pipelines, and dual-domain fusion.

Chapter 2

Literature Survey

2.1 Computer Vision for Artistic Reference Analysis

Artists have historically relied on manual grid methods — drawing a regular grid over a reference image and a matching grid on the target canvas — to transfer proportions accurately across different scales. While effective in principle, this approach is static: it cannot adapt when the reference and canvas are of different aspect ratios, and it provides no machine-readable spatial information that could drive measurement feedback. The emergence of mobile computer vision has opened the possibility of replacing these manual grids with active, measurement-aware analysis directly on a reference photograph.

The ArtGrid system draws on four distinct bodies of research and technology: (i) general object and face detection using convolutional neural networks, specifically the YOLO family; (ii) classical geometric face landmark regression via the dlib library; (iii) salient subject segmentation using encoder-decoder architectures; and (iv) perceptual colour-science models suited to pigment-based artistic practice. This chapter surveys the foundational work in each area, positions ArtGrid’s contributions relative to existing approaches, and concludes with the project timeline.

2.2 Foundational Technologies

2.2.1 Object and Face Detection: The YOLO Family

YOLO [9] is a single-stage detector family suited to realtime throughput; YOLOv8 spans compact YOLOv8n (**general objects** in ArtGrid) to YOLOv8x6 with a P6 pyramid for the **anime-face** head (Section 2.3.2). Both checkpoints are exported to ONNX and served with ONNX Runtime [7] on the cloud GPU (CUDA or TensorRT).

2.2.2 Facial Landmark Detection: dlib and Ensemble Regression Trees

dlib [8] combines a HOG face detector with Kazemi and Sullivan’s 68-point ERT shape predictor [15]; landmark chords (eyes, nose, mouth, jaw) ground the centimetre grid on human portraits. The ERT stage runs on CPU in parallel with the YOLO GPU branch.

2.2.3 Image Segmentation: U²-Net

U²-Net [10] uses nested residual U-blocks for salient foreground estimation; ArtGrid employs the portrait variant up to 800 px long edge to produce an alpha mask for the Background Remover screen.

2.2.4 Perceptual Colour Spaces: Oklab

Display spaces (sRGB, HSL) are not perceptually uniform; Oklab [16] improves hue linearity vs. CIELAB for display-referred inputs. ArtGrid uses Oklab L for greyscale and (L, a, b) for tap sampling.

2.2.5 Kubelka–Munk Paint Mixing Theory

The Kubelka–Munk (K–M) theory [14] describes the optical behaviour of paint layers in terms of two wavelength-dependent coefficients: K (absorption) and S (scattering). For a single opaque layer, the reflectance R at a given wavelength is related to these coefficients by:

$$R = 1 + \frac{K}{S} - \sqrt{\left(\frac{K}{S}\right)^2 + 2\frac{K}{S}} \quad (2.1)$$

When two pigments are mixed in a ratio $c : (1 - c)$, the mixture coefficients are:

$$K_{\text{mix}} = c \cdot K_1 + (1 - c) \cdot K_2, \quad S_{\text{mix}} = c \cdot S_1 + (1 - c) \cdot S_2 \quad (2.2)$$

and the resulting reflectance is computed by substituting K_{mix} and S_{mix} into Equation 2.1. ArtGrid’s K–M solver operates on sampled RGB values: it converts the target colour into per-channel K/S ratios, then solves a least-squares optimisation to find the mixing ratio c that minimises the perceptual error (in Oklab $L^*a^*b^*$) between the predicted mixture reflectance and the target.

2.2.6 Edge Detection, Kuwahara Filtering, and Perspective Correction

The playground uses Canny edges [17] with an adjustable threshold for structural emphasis, and Kuwahara smoothing [18] for edge-preserving, painterly

previews. Perspective correction fits a homography $\mathbf{H}$ by DLT from four user corners [1] and warps the image in native C++ for interactive preview.

2.3 Animated Face Detection Research

2.3.1 Challenges of Detecting Anime-Style Faces

Anime and illustrated character faces differ from photorealistic human faces in several ways that confound standard detectors:

- **Extreme simplification:** Eyes may occupy 30–40% of the face area; noses and mouths are often reduced to minimal strokes.
- **High stylistic variance:** Line art, flat colour, painterly, and 3D-rendered styles all appear in the same domain.
- **Non-photorealistic lighting:** Specular highlights, cel shading, and flat shadow fills do not match the gradient transitions that HOG descriptors and photorealistic CNN features expect.
- **Partial occlusion:** Foreground elements (hair, accessories) frequently occlude large portions of the face.

Classical HOG-based detectors (including dlib) fail on these faces. Cascade classifiers trained on anime data (e.g., `lbpcascade_animeface`) achieve high recall only for near-frontal, clean-line faces and produce large numbers of false positives on complex scenes.

2.3.2 YOLOv8x6 Anime-Face Model

For ArtGrid, a YOLOv8x6 model was trained specifically on anime-style face data sourced from Pixiv. The training set consists of a dataset of 10,000 annotated anime pictures collected exclusively from Pixiv, and the test set encompasses pictures selected from the daily rankings on Pixiv. The dataset split follows a 70% / 20% / 10% (train / validation / test) ratio.

Training used 300 epochs at 1280×1280 px, batch size 16, and the default SGD optimiser with Comet ML monitoring; the verbatim Ultralytics entry point appears in Listing 1 (Appendix 7.5).

An important training observation: when early stopping was applied, the model achieved a higher mAP₅₀₋₉₅ (≈ 0.57 on the validation set) but low detection confidence (≈ 0.5). Training the full 300 epochs without early stopping lowered the mAP₅₀₋₉₅ to ≈ 0.52 but raised confidence to ≈ 0.8 per detection — a more useful operating point for an interactive application where false negatives are more disruptive than minor accuracy loss.

The final model performance on the held-out Pixiv test set is:

Table 2.1: Anime-Face Detector Performance on Pixiv Test Set

Metric	Precision	Recall	mAP50	mAP50-95
YOLOv8x6 (ArtGrid)	0.957	0.924	0.955	0.534

Inference speed on an RTX A4000-class online GPU is 81.9 ms per image at 1280×1280 resolution; deployment uses ONNX in artgrid-ml-service via ONNX Runtime [7]. Quantitative comparison against the prior YOLOv5-anime baseline [19] (and the broader anime-face literature exemplified by the Kaggle Anime Face Dataset [20]) is reported exclusively in Chapter 6 to avoid duplicating numerical results outside the Results chapter.

2.4 Large Language Models and SSE Streaming

ArtGrid includes an AI Artist chat feature powered by the Gemini API. Rather than calling the Gemini API directly from the Android client — which would require embedding the API key in the application binary — a dedicated Go proxy service (artgrid-ai-proxy) receives the chat request, forwards it to Gemini, and returns the response as a Server-Sent Events (SSE) stream [13].

SSE is a W3C-standardised protocol for unidirectional, server-to-client streaming over HTTP. Each event is a newline-delimited text frame prefixed with `data:`, allowing the client to display partial completions as tokens arrive. In ArtGrid, the Android client parses the SSE stream using a custom `parseSseStream` function on a raw `OkHttp` response body [3], since Retrofit would buffer the entire stream before returning.

2.5 Mobile Architecture and Microservices

ArtGrid’s Android client is built with Jetpack Compose, Google’s declarative UI toolkit for Android, which replaces the traditional XML layout system with composable functions. Local data persistence for reference images and progression stages uses Room [2], a type-safe SQLite abstraction. The native C++ image library is compiled via the Android NDK [4] using CMake and loaded at runtime through JNI.

The backend follows a microservice architecture [21], with services separated by concern: authentication and identity, machine learning inference, and AI chat proxying. Each service has independent deployment and scaling properties, which is particularly important for the ML service, which requires GPU resources unavailable to the lighter authentication and proxy services.

2.6 Related Work and Positioning

Table 2.2 summarises how ArtGrid’s contributions compare with the most relevant existing systems and tools.

Table 2.2: Existing Systems vs. ArtGrid Contributions

Area	Existing Work	ArtGrid Contribution
Unified domain pipeline multi-face	No publicly available tool fuses human landmark detectors with anime-face YOLO in a single endpoint	ArtGridFaceDomainFusion: parallel dlib + YOLOv8 heads, cross-head NMS, domain tagging
Mobile colour science	Standard apps report sRGB or HSL	Oklab [16] perceptual sampling, Kelvin temperature, K–M mix recipe [14]
Spatial art tools	Static printed grids; desktop-only tools	Real-time cm-grid calibrated to physical paper size; A–B ruler; paper-to-paper mapping
LLM chat in mobile	API key embedded in app	Dedicated SSE Go proxy [13]; Gemini key server-side
Identity security	Plaintext storage or Firebase	AES-256-GCM encrypted PII fields in PostgreSQL [6]

2.7 Project Timeline

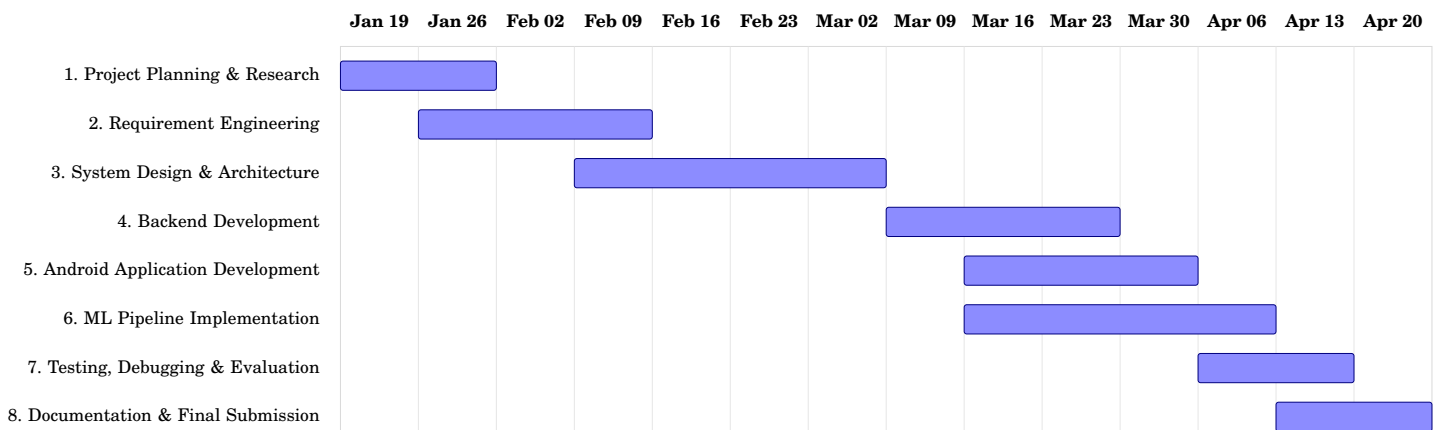


Figure 2.1: Gantt Chart (19 January 2026 – 20 April 2026)

Chapter 3

Requirement Engineering

3.1 Functional Requirements

S No.	Requirement	Description
1	Google Sign-In Authentication	The system shall authenticate users via Google Sign-In ID token exchange and issue a JWT access token and a refresh token stored on the device.
2	Token Refresh	The system shall silently refresh the JWT access token using the refresh token before API calls fail with 401 Unauthorized.
3	Reference Image Import	The user shall be able to import a reference photograph from the device camera or gallery. The image is stored locally in Room as a ReferenceAsset with URI, dimensions, and thumbnail.
4	Unified Human and Animated Face Detection	The system shall invoke the /infer/face_unified endpoint, which applies dlib HOG detection and YOLOv8x6 ONNX detection in parallel, fuses the results via ArtGridFaceDomainFusion, and returns a ranked list of detections with a domain tag (human or anime).
5	Human Face Detection	The system shall send the reference image to artgrid-ml-service and receive bounding boxes with face landmarks for each detected human face.
6	Object Detection	The system shall send the reference image to the /infer/objects endpoint and display labelled bounding boxes for detected objects overlaid on the reference image.

S No.	Requirement	Description
7	Background Segmentation	The system shall send the reference image to the /infer/segment endpoint and display a binary segmentation mask separating the foreground subject from the background.
8	Cm-Grid Overlay	The system shall overlay a metric grid, calibrated to the user-specified physical canvas size (e.g., A4: 21×29.7 cm, A3: 29.7×42 cm, or custom dimensions), on the reference image on screen.
9	A–B Ruler Measurement	The user shall be able to define two points on the reference image by tapping; the system shall display the pixel distance and its equivalent centimetre distance on the chosen canvas.
10	Oklab Colour Sampler	The user shall be able to tap any pixel on the reference image to sample its colour and receive the value expressed in Oklab, HSL, and approximate Kelvin temperature.
11	K–M Paint-Mix Solver	Given a sampled target colour, the system shall compute a two-pigment mixing ratio using the Kubelka–Munk model and display the suggested blend.
12	Dominant Palette Extraction	The system shall extract up to six dominant colours from the reference image using palette quantisation and display them as swatches.
13	Image Playground Filters	The system shall provide on-device C++ filters on the imported reference: Oklab greyscale, tonal heatmap, grey-world white balance, linear invert, Kuwahara edge-preserving filter, and edge extraction with adjustable λ sensitivity.
14	Perspective Correction	The user shall be able to select four corner points on a distorted reference and receive a deskewed image via homographic warp.
15	Trace Mode	The system shall superimpose a live camera feed as a transparent underlay beneath the reference grid, allowing the artist to use the device as a tracing aid.
16	Paper Mapping	The system shall map the reference image proportionally onto the dimensions of the chosen physical canvas, showing which sub-regions correspond to which parts of the reference.
17	AI Artist Chat	The user shall send text or the current reference image to the AI proxy, which streams a Gemini-generated response token-by-token using Server-Sent Events via artgrid-ai-proxy.

S No.	Requirement	Description
18	Reference History	The system shall maintain a scrollable local history of all imported reference images stored in Room, showing thumbnails, import date, and tags.
19	Progression Comparator	The user shall be able to define a named progression, add chronological stage images to it, and compare stages side-by-side with a slider.
20	ML Health Gate	The system shall poll the ML service health endpoint at app launch; if unavailable, ML feature cards shall display a “Service Offline” indicator rather than silently failing.

3.2 Non-Functional Requirements

S No.	Requirement	Description
1	Offline Availability	All on-device filters and local data operations shall function without network connectivity. Only ML inference and AI chat require network access.
2	Security — Encrypted PII	User google_sub and email shall be encrypted with AES-256-GCM via pgcrypto before storage in PostgreSQL. No plaintext PII shall appear in database logs.
3	Security — API Key Isolation	The Gemini API key shall never reside in the Android application binary. All Gemini requests shall be proxied through artgrid-ai-proxy using an hourly HMAC-SHA256 device token.
4	Security — HTTPS Only	All client-server communication shall use HTTP-S/TLS. No raw image data or credentials shall be transmitted over unencrypted channels.
5	ML Inference Latency	The unified face detection endpoint shall return results within 5 seconds for a 1280×1280 input on the deployed GPU instance.
6	On-Device Filter Latency	All C++ native filters shall complete within 500 ms on a mid-range Android device (Snapdragon 720G class or higher).
7	Scalability	The ML service shall be deployable on multiple GPU instances behind a load balancer. Auth and proxy services shall be stateless and horizontally scalable.

S No.	Requirement	Description
8	Maintainability	Each of the three backend services shall be independently deployable with its own Dockerfile and health endpoint. A failure in one service shall not crash the others.
9	Data Privacy	The ML service shall not log or persist raw user images. Inference inputs shall be processed in memory only.
10	Usability	The application shall provide distinct Loading, Success, Empty, and Error states for all ML operations so the user always understands the application state.

3.3 System Requirements

3.3.1 Software Requirements

Table 3.3: Software Requirements for ArtGrid

Component	Version / Constraint
Android SDK	API Level 26 minimum; API Level 34 target
Kotlin + Jetpack	Kotlin 1.9+; Compose BOM 2024.x
Room	2.6+ (local SQLite persistence)
OkHttp	4.12+ (Retrofit HTTP + raw SSE client)
Android NDK	r26+; CMake 3.22+ (C++ native build)
Python	3.11+ (artgrid-ml-service)
FastAPI	0.110+; Uvicorn ASGI server
ONNX Runtime	1.17+ with CUDA 12 execution provider
dlib	19.24+ compiled with CUDA support
Go	1.22+ (artgrid-auth-service, artgrid-ai-proxy)
PostgreSQL + pgcrypto	PostgreSQL 15+ with pgcrypto extension
Docker Compose	Docker Engine 25+; Compose V2

3.3.2 Hardware Requirements

Table 3.4: Hardware Requirements for ArtGrid

Component	Specification
Android Device	ARMv8-A 64-bit; 3 GB RAM min; Android 8.0+; rear camera
Development Machine	8-core CPU; 16 GB RAM; 50 GB SSD; Windows / macOS / Linux
GPU VM (ML service)	NVIDIA GPU ≥ 8 GB VRAM, CUDA 12 (e.g., T4/L4 on GCP; RTX 3080/4090 on RunPod)
Auth / Proxy Server	2-core CPU; 2 GB RAM; no GPU required

3.4 Use Cases

3.4.1 Use Case Diagram

Figure 3.1 shows the Use Case diagram for ArtGrid. The primary actors are the **User** (the artist), the **artgrid-auth-service**, the **artgrid-ml-service**, and the **artgrid-ai-proxy**.

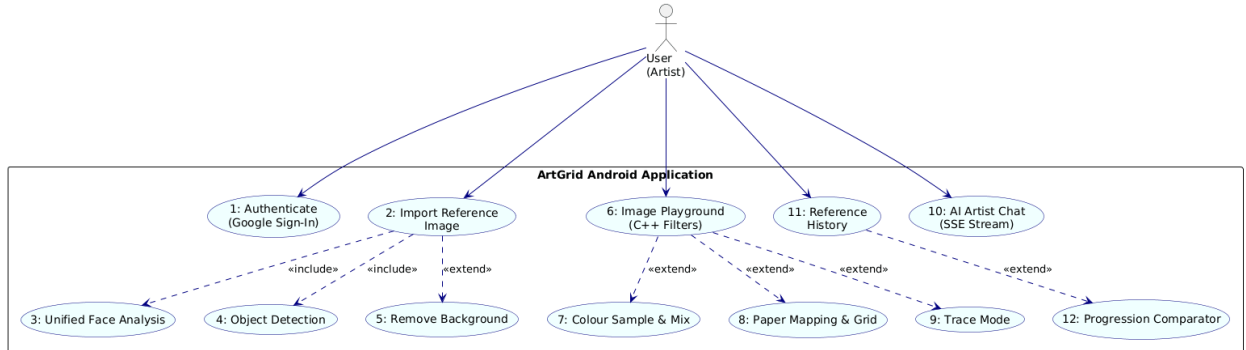


Figure 3.1: Use Case Diagram

3.4.2 Use Case Descriptions

S No.	Use Case	Actors	Description
1	Authenticate	User, artgrid-auth-service	Google ID token exchanged for JWT. Maps to S.No. 1, 2 (FR).
2	Import Reference Image	User	Photo captured or selected; saved as ReferenceAsset in Room. Maps to S.No. 3 (FR).

S No.	Use Case	Actors	Description
3	Unified Face Analysis	User, artgrid-ml-service	ArtGridFaceDomainFusion returns human and anime detections. Maps to S.No. 4, 5 (FR).
4	Object Detection	User, artgrid-ml-service	YOLOv8n returns labelled bounding boxes. Maps to S.No. 6 (FR).
5	Remove Background	User, artgrid-ml-service	U ² -Net returns foreground mask. Maps to S.No. 7 (FR).
6	Image Play-ground	User	C++ on-device filters applied to reference. Maps to S.No. 13, 14 (FR).
7	Colour Sample and Mix	User	Tap to sample Oklab colour; view K-M mix recipe. Maps to S.No. 10, 11, 12 (FR).
8	Paper Mapping and Cm-Grid	User	Canvas size selected; metric grid and paper map rendered. Maps to S.No. 8, 9, 16 (FR).
9	Trace Mode	User	Live camera underlay activated for proportional tracing. Maps to S.No. 15 (FR).
10	AI Artist Chat	User, artgrid-ai-proxy	SSE-streamed Gemini response delivered to chat panel. Maps to S.No. 17 (FR).
11	Reference History	User	Local Room history of imports browsed. Maps to S.No. 18 (FR).
12	Progression Comparator	User	Named progression created; stages compared with slider. Maps to S.No. 19 (FR).

Chapter 4

System Design

4.1 System Architecture

ArtGrid follows a client-server architecture with three independently deployable backend microservices. The Android client communicates with each service over HTTPS. The auth service and AI proxy are lightweight Go services that run on standard CPU instances, while the ML service is deployed on a cloud GPU instance (Google Cloud Compute Engine or RunPod) to handle the computational cost of the YOLOv8x6 ONNX inference. Figure 4.1 shows the complete system: the Android client on the left communicates upward to the three microservices. The ML service connects to the ONNX Runtime inference engine with the dlib and YOLOv8 models loaded. The AI proxy connects to the external Gemini API, and all inter-service communication uses JSON over HTTPS.

4.2 Module Design

4.2.1 Android Client

The Android application is structured into four logical layers:

- **UI Layer** (ui/): Jetpack Compose screens and navigation graph. Screens include: HomeScreen, AnalysisResultScreen, FaceStudioScreen, ObjectLocatorScreen, BackgroundRemoverScreen, ImagePlaygroundScreen, ColourPaletteScreen, PaperMappingScreen, TraceModeScreen, ChatScreen, ReferenceHistoryScreen, ProgressionScreen.
- **Domain Layer** (domain/): Use case classes and repository interfaces. Provides the logic boundary between UI and data.
- **Data Layer** (data/): Repository implementations, Room DAOs, Retrofit service interfaces, and OkHttp SSE client.

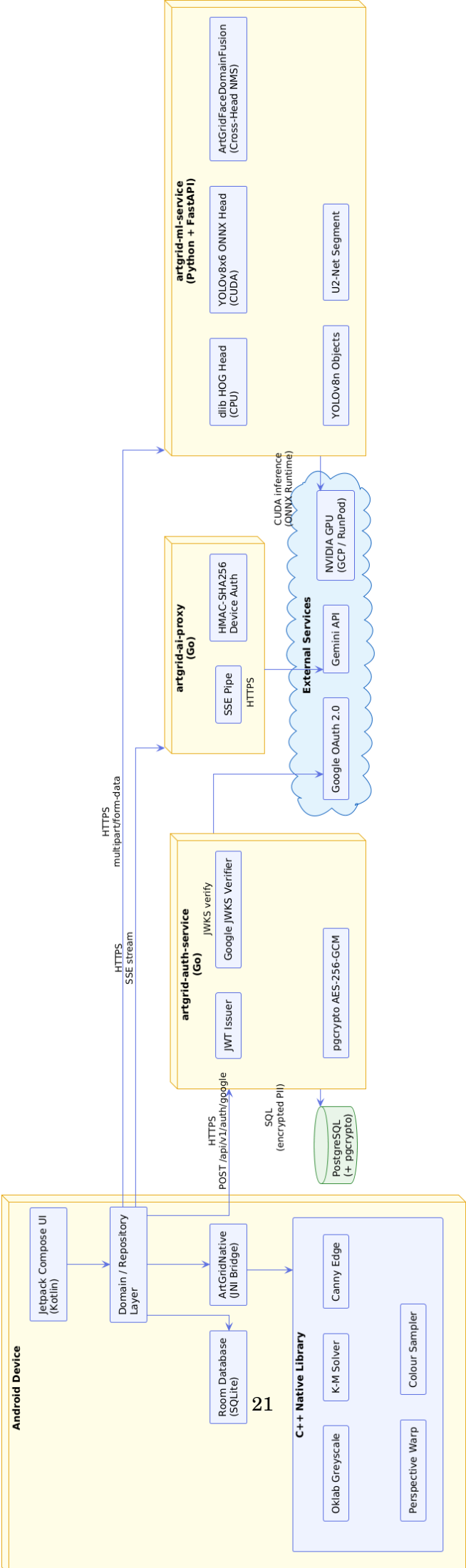


Figure 4.1: System Architecture

- **Native Bridge** (nativebridge/ + cpp/): Kotlin ArtGridNative object exposing C++ functions via JNI. The native library is compiled via CMake and the Android NDK.

4.2.2 artgrid-auth-service

The authentication service is written in Go with the Chi router. It exposes three endpoints:

- POST /api/v1/auth/google — Validates a Google ID token with Google’s public JWKS endpoint, creates or retrieves the user record in PostgreSQL, and returns a short-lived JWT access token (15 min) and a long-lived refresh token (30 days).
- POST /api/v1/auth/refresh — Validates the refresh token and issues a new access token.
- GET /health — Returns service liveness status.

User google_sub and email fields are encrypted with AES-256-GCM via the PostgreSQL pgcrypto extension before insertion. The encryption key is injected as an environment variable and never hard-coded.

4.2.3 artgrid-ml-service

The machine learning service is written in Python with FastAPI. Model loading occurs once at application startup via the FastAPI lifespan event, preventing per-request cold starts. The service exposes the following inference endpoints:

- POST /infer/face — dlib HOG detection and face landmark regression for human faces.
- POST /infer/face_unified — ArtGridFaceDomainFusion: parallel dlib + YOLOv8x6 ONNX inference, cross-head NMS, domain tagging. Returns a unified list of FaceResult objects with domain: "human" | "anime".
- POST /infer/objects — YOLOv8n ONNX for general object detection.
- POST /infer/segment — U²-Net for salient subject segmentation.
- GET /health — Returns service liveness and model loading status.

All endpoints accept multipart/form-data with the image file and a Bearer JWT for authentication. ONNX Runtime is used for all model inference, enabling GPU acceleration via the CUDA execution provider without a PyTorch dependency at inference time.

4.2.4 artgrid-ai-proxy

The AI proxy is a thin Go service. It authenticates incoming requests using an HMAC-SHA256 device token (X-Device-Token) keyed per hour. On receiving a valid chat request, it constructs a Gemini API call, sets alt=sse to enable streaming, and pipes the Server-Sent Events response back to the Android client. The Gemini API key is stored only as a server-side environment variable.

4.2.5 Native C++ Image-Processing Library

The C++ native library (libartgrid.so) is compiled with CMake via the Android NDK and exposes functions to Kotlin through JNI. The library implements the following operations, each with its mathematical basis:

Oklab Greyscale Conversion

The Oklab colour space maps sRGB values to a perceptually uniform space (L, a, b) . For greyscale conversion, only the L component is used. The conversion from linear sRGB (r_l, g_l, b_l) proceeds as:

$$\begin{pmatrix} l \\ m \\ s \end{pmatrix} = \begin{pmatrix} 0.4122214708 & 0.5363325363 & 0.0514459929 \\ 0.21119034982 & 0.6806995451 & 0.1073969566 \\ 0.0883024619 & 0.2817188376 & 0.6299787005 \end{pmatrix} \begin{pmatrix} r_l \\ g_l \\ b_l \end{pmatrix} \quad (4.1)$$

$$L = 0.2104542553 l'^{1/3} + 0.7936177850 m'^{1/3} - 0.0040720468 s'^{1/3} \quad (4.2)$$

where $l' = l^{1/3}$, $m' = m^{1/3}$, $s' = s^{1/3}$. The greyscale output pixel is $G = \text{clip}(L, 0, 1) \times 255$.

Kubelka–Munk Paint-Mix Solver

The K–M solver operates per RGB channel. For a single pigment with reflectance $R \in [0, 1]$, the absorption-to-scattering ratio is:

$$\frac{K}{S} = \frac{(1-R)^2}{2R} \quad (4.3)$$

For a two-pigment mix at ratio $c : (1-c)$:

$$\left(\frac{K}{S}\right)_{\text{mix}} = c \cdot \frac{K_1}{S_1} + (1-c) \cdot \frac{K_2}{S_2} \quad (4.4)$$

The predicted mix reflectance is recovered from Equation 4.3 inverted:

$$R_{\text{mix}} = 1 + \frac{K_{\text{mix}}}{S_{\text{mix}}} - \sqrt{\left(\frac{K_{\text{mix}}}{S_{\text{mix}}}\right)^2 + 2 \frac{K_{\text{mix}}}{S_{\text{mix}}}} \quad (4.5)$$

The optimal ratio c^* is found by minimising the Oklab perceptual distance ΔE between the predicted mix and the target colour over $c \in [0, 1]$ using a golden-section search.

Canny Edge Extraction

Edge extraction follows the Canny pipeline. Let I be the input greyscale image. After Gaussian smoothing with kernel σ , the gradient magnitude at each pixel is:

$$M(x, y) = \sqrt{G_x(x, y)^2 + G_y(x, y)^2} \quad (4.6)$$

where G_x and G_y are the horizontal and vertical Sobel responses. Non-maximum suppression thins the edges to single-pixel width, and hysteresis thresholding with user-adjustable λ determines the final binary edge map.

Perspective Correction (Homography)

Given four user-selected source corners $\{\mathbf{p}_i\}$ and the target rectangle corners $\{\mathbf{q}_i\}$, the homography $\mathbf{H}$ is estimated by Direct Linear Transform (DLT):

$$\tilde{\mathbf{q}}_i \sim \mathbf{H} \tilde{\mathbf{p}}_i, \quad \mathbf{H} \in \mathbb{R}^{3 \times 3}, \quad \det(\mathbf{H}) \neq 0 \quad (4.7)$$

The DLT constructs a linear system $\mathbf{A} \text{vec}(\mathbf{H}) = \mathbf{0}$ and solves via SVD. The warp is applied with bilinear interpolation.

Colour Temperature (Kelvin Approximation)

For a sampled sRGB colour, an approximate correlated colour temperature T in Kelvin is computed using the McCamy formula on the CIE xy chromaticity coordinates (x, y) derived from the XYZ transform of the sRGB input:

$$n = \frac{x - 0.3320}{0.1858 - y}, \quad T = 449n^3 + 3525n^2 + 6823.3n + 5520.33 \quad (4.8)$$

4.3 Storage Design

4.3.1 PostgreSQL Schema

Table 4.1: PostgreSQL users Table (artgrid-auth-service)

Column	Type	Notes
id	UUID PK	Auto-generated, primary identifier
google_sub	TEXT	AES-256-GCM encrypted via pgcrypto
email	TEXT	AES-256-GCM encrypted
created_at	TIMESTAMPTZ	Insertion timestamp
updated_at	TIMESTAMPTZ	Last update
deleted_at	TIMESTAMPTZ	Soft delete (nullable)

4.3.2 Room Database Schema

Table 4.2: Room Database Tables (ArtGrid Android)

Column	Type	Notes
Table: reference_assets		
id	INTEGER PK	Auto-increment
uri_local	TEXT	Content URI of the imported image
width_px	INTEGER	Original image width
height_px	INTEGER	Original image height
thumbnail_path	TEXT	Path to generated thumbnail
tags	TEXT	Comma-separated user tags
created_at	INTEGER	Unix epoch ms
Table: progressions		
id	INTEGER PK	Auto-increment
title	TEXT	User-defined progression name
created_at	INTEGER	Unix epoch ms
Table: progression_stages		
id	INTEGER PK	Auto-increment
progression_id	INTEGER FK	Foreign key to progressions.id
stage_index	INTEGER	Ordering index
image_uri	TEXT	Content URI of stage photograph
label	TEXT	Optional stage label
created_at	INTEGER	Unix epoch ms

4.4 Data Flow Diagrams

4.4.1 Level 0 DFD

Figure 4.2 shows the context-level (Level 0) Data Flow Diagram for ArtGrid. The system is represented as a single process receiving inputs from the external *User* actor and communicating with external systems: Google OAuth, artgrid-ml-service, and the Gemini API.

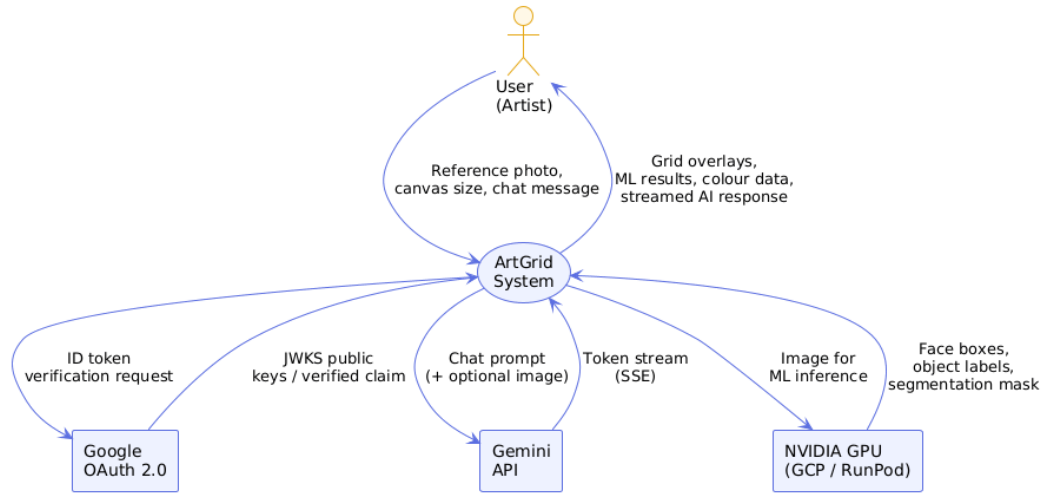


Figure 4.2: Data Flow Diagram Level 0

4.4.2 Level 1 DFD

Figure 4.3 expands the system into its five internal processes: Authentication, Image Analysis, On-Device Processing, AI Chat, and Data Management. Data stores D1 (PostgreSQL Users), D2 (Room References), and D3 (Room Progressions) are shown with their data flows.

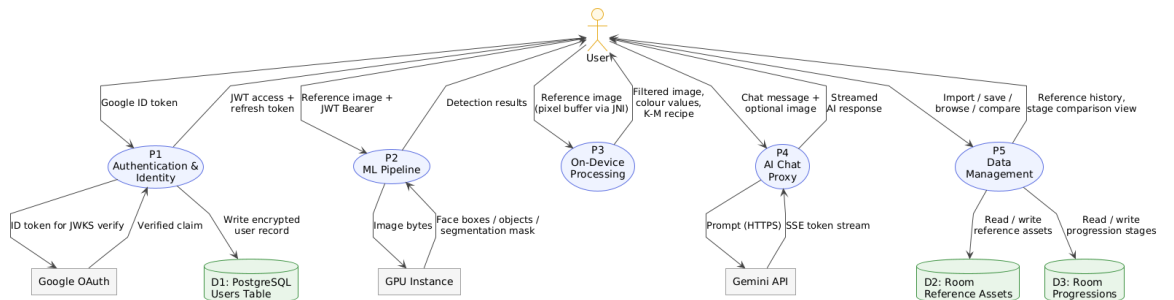


Figure 4.3: Data Flow Diagram Level 1

4.5 Process Flowchart

Figure 4.4 illustrates the primary user workflow in ArtGrid: from app launch and authentication through reference import, ML analysis, and the branching paths to on-device tools, AI chat, or data persistence.

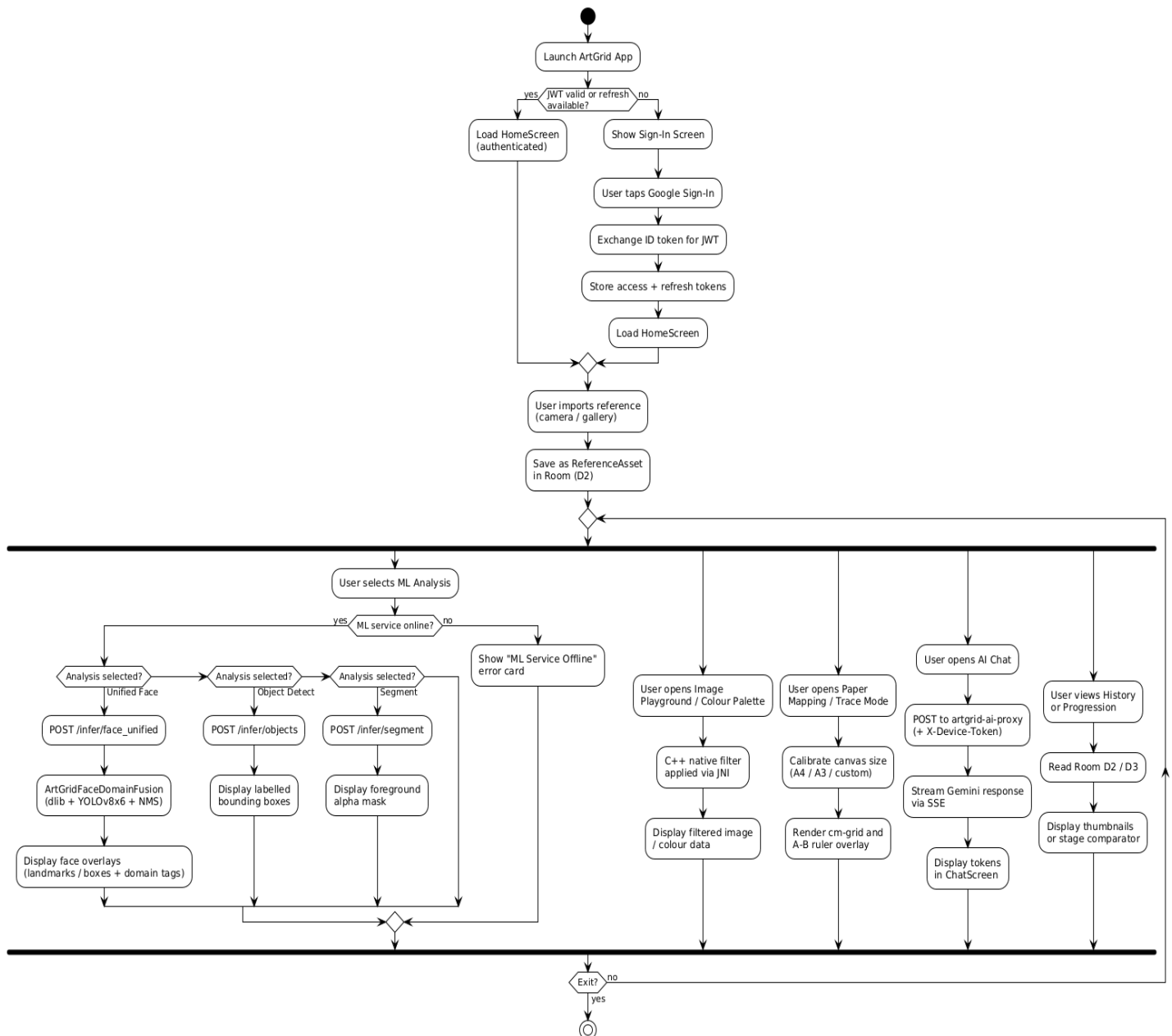


Figure 4.4: Flowchart

4.6 Sequence Diagram

Figure 4.5 shows the interaction sequence for the three primary flows: (1) Google authentication and JWT issuance, (2) unified face analysis via artgrid-ml-service, and (3) SSE-streamed AI chat via artgrid-ai-proxy.

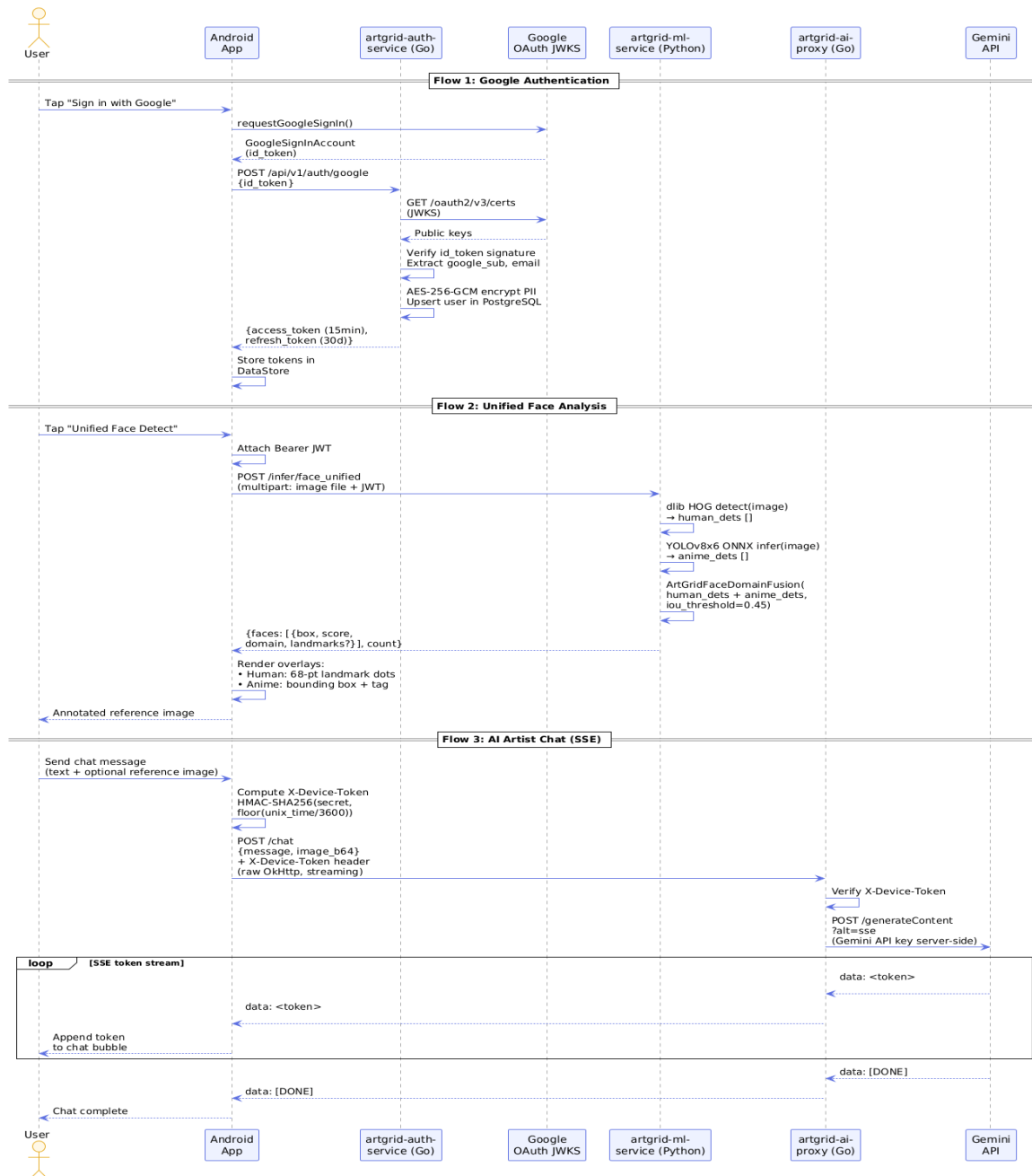


Figure 4.5: Sequence Diagram

4.7 Component Diagram

Figure 4.6 shows the component-level view of ArtGrid, illustrating the internal packages of the Android client, the three backend services, and their external dependencies (ONNX Runtime, dlib, Room, PostgreSQL, Gemini API).

4.8 State Machine Diagram

Figure 4.7 shows the two state machines that govern ArtGrid's runtime behaviour: the ML Feature State Machine (per analysis card: Loading → Success / Empty / Error, with retry from Error) and the Authentication State Machine (Unauthenticated → Signing In → Authenticated → Token Expired → Refreshing).

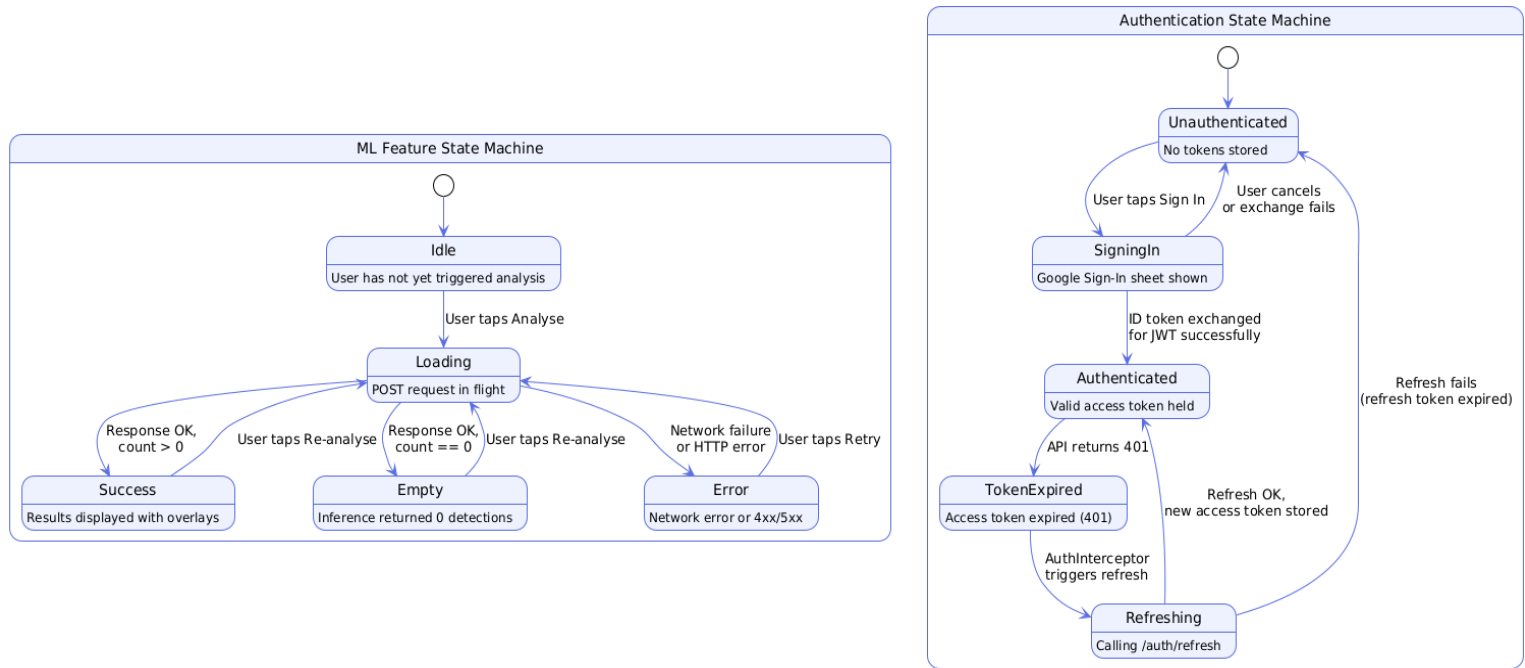


Figure 4.7: State Machine Diagram

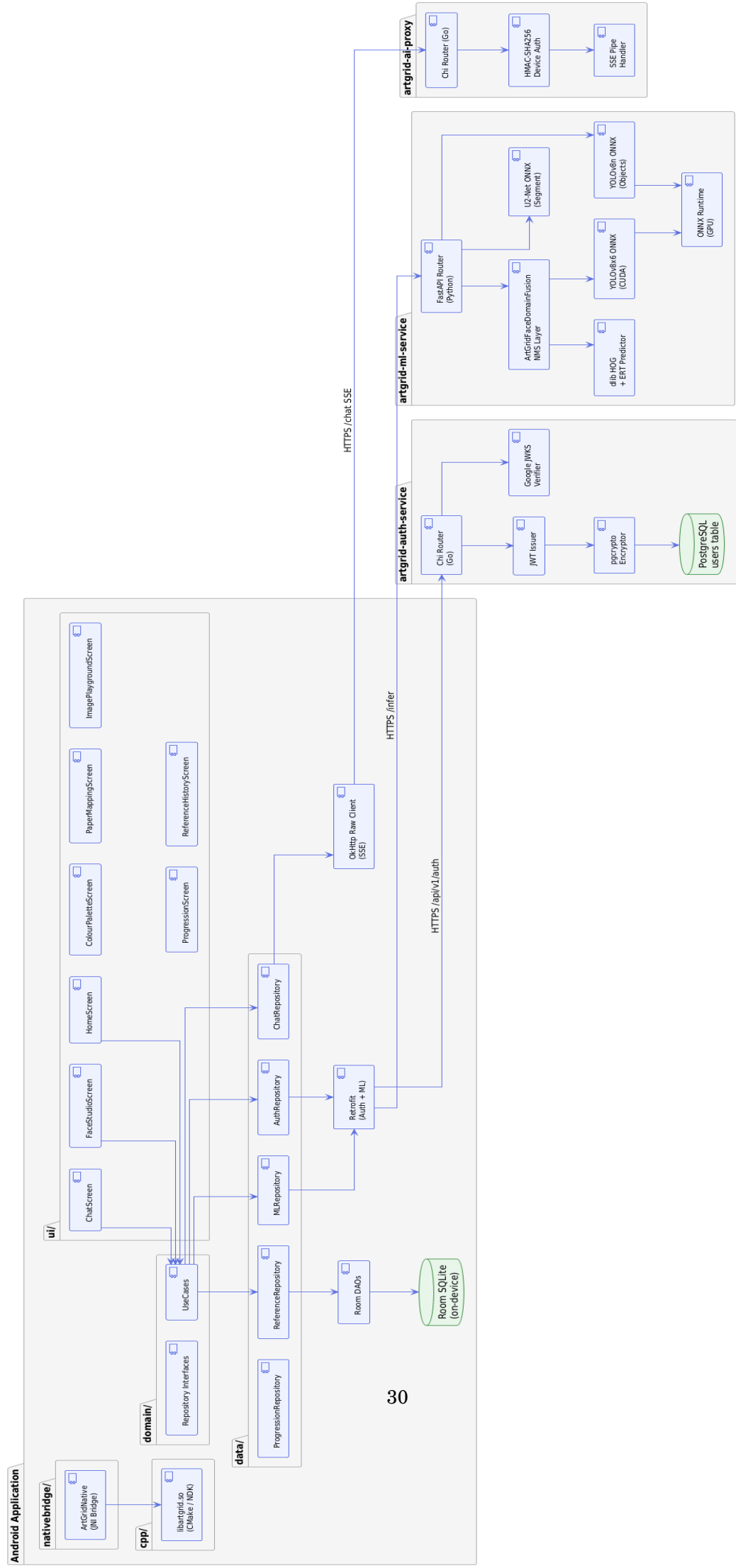


Figure 4.6: Component Diagram

Chapter 5

Implementation

5.1 Architecture Overview

ArtGrid’s implementation spans four major components, developed in parallel and integrated through well-defined interfaces. The Android client (Kotlin, Jetpack Compose) communicates with two Go services and one Python service over HTTPS. The C++ native library is compiled into a shared object and loaded at app startup via JNI. Each component is described in order of contribution priority, beginning with the most original engineering work. Systematic validation (functional coverage, integration paths, timed performance, and failure handling) is summarised in Chapter 6.

5.2 ArtGridFaceDomainFusion: Unified Human and Animated Face Detection

The ArtGridFaceDomainFusion pipeline is the primary engineering contribution of this project. It is the only publicly available inference endpoint that simultaneously applies a classical face landmark detector for photorealistic portraits and a deep convolutional detector for anime-style illustrations, then fuses the results into a single ranked output.

5.2.1 Design Rationale

Existing tools treat human and animated faces as separate problems. A single-pass approach is necessary for ArtGrid because the centimetre measurement workflow — grid overlays, chord distances, paper mapping — must operate identically on both reference types. The fusion layer must suppress duplicate detections arising from a face that is partially detected by both heads, and must tag each surviving detection with a domain label so the UI can render appropriate landmark overlays.

5.2.2 Parallel Inference Architecture

The endpoint runs two detection heads concurrently:

1. **dlib HOG head** (CPU): The dlib frontal face HOG detector localises human face bounding boxes. For each box, the ERT 68-point shape predictor (pre-trained `shape_predictor_68_face_landmarks.dat`) regresses the landmark keypoints. The face area confidence is estimated from the HOG response score.
2. **YOLOv8x6 ONNX head** (GPU): The trained anime-face ONNX model runs via ONNX Runtime with the CUDA execution provider. The model was exported from its `.pt` checkpoint using:

Algorithm 1 Exporting YOLOv8x6 to ONNX for deployment

Require: Trained weights file `best.pt` for `yolov8x6-animeface`

Ensure: ONNX file `best.onnx` suitable for ONNX Runtime

- 1: `model ← YOLO(yolov8x6-animeface/weights/best.pt)`
 - 2: `model.export(format=onnx, imgsz=1280, opset=17, simplify=true)`
 - 3: output file: `yolov8x6-animeface/weights/best.onnx`
-

5.2.3 API Endpoint Implementation

The endpoint is registered in `artgrid-ml-service/router.py` as a FastAPI route; full lifespan wiring and the request handler appear as Algorithm 3 in Appendix 7.5.

5.2.4 Cross-Head NMS Fusion

The pseudocode for `ArtGridFaceDomainFusion` is given in Algorithm 2. Detections from both heads are pooled into a single candidate list. Cross-head NMS is applied with a configurable IoU threshold (default 0.45) so that if a human and an anime detection overlap significantly, the higher-confidence detection is retained and the other is suppressed.

Algorithm 2 ArtGridFaceDomainFusion cross-head NMS

Require: image I , IoU threshold τ (default 0.45)

Ensure: list kept of detections, each with domain $\in \{\text{human}, \text{anime}\}$

```
1: human_dets  $\leftarrow$  DLIB_HOG(det(I))            $\triangleright$  boxes, score, landmarks,
   domain="human"
2: anime_dets  $\leftarrow$  YOLOV8_ONNX(det(I))        $\triangleright$  boxes, score, domain="anime"
3: candidates  $\leftarrow$  human_dets  $\parallel$  anime_dets
4: sort candidates by score descending
5: kept  $\leftarrow []$ ; suppressed  $\leftarrow \emptyset$ 
6: for each index  $i$ , detection  $\text{det}_i$  in candidates (in order) do
7:   if  $i \in$  suppressed then
8:     continue
9:   end if
10:  append  $\text{det}_i$  to kept
11:  for each  $j > i$  with  $\text{det}_j$  in candidates do
12:    if  $\text{IoU}(\text{det}_i.\text{box}, \text{det}_j.\text{box}) \geq \tau$  then
13:      add  $j$  to suppressed
14:    end if
15:  end for
16: end for
17: return kept
```

5.3 C++ Native Image-Processing Library

The C++ library (libartgrid.so) provides all on-device image processing. It is built via CMake as part of the Android Gradle build and linked into the app via JNI. Each function in ArtGridNative.kt maps to a corresponding native method.

5.3.1 JNI Bridge

The Kotlin bridge loads libartgrid.so and declares native entry points for greyscale, K-M, Canny, perspective correction, and colour sampling; the declaration block is given as Algorithm 4 in Appendix 7.5.

5.3.2 Oklab Greyscale (C++ Implementation)

The C++ implementation converts each pixel through the LMS cone response matrix and extracts the L component as described in Chapter 4; the per-pixel pipeline is Algorithm 5 in Appendix 7.5.

5.3.3 K-M Paint-Mix Solver

The solver performs a golden-section search over $c \in [0, 1]$ to minimise the Oklab perceptual error ΔE between the predicted mix reflectance and the target colour, evaluating the loss at each candidate using Equations 4.3–4.5 from Chapter 4. The search structure is Algorithm 6 in Appendix 7.5.

5.4 Android Application Implementation

5.4.1 Navigation Graph and Route Table

ArtGrid uses Jetpack Compose Navigation with a single-activity architecture. Routes are defined as sealed class constants, and all screens share a single NavHost. The primary routes are:

Table 5.1: ArtGrid Navigation Routes

Route	Screen
home	HomeScreen — import reference, ML cards, FAB chat
analysis_result	AnalysisResultScreen — filter playground on result
face_studio	FaceStudioScreen — landmark overlays
object_locator	ObjectLocatorScreen — object bounding box display
background_remover	BackgroundRemoverScreen — segmentation mask
image_playground	ImagePlaygroundScreen — C++ filter chain
colour_palette	ColourPaletteScreen — Oklab sampler, K–M mixer
paper_mapping	PaperMappingScreen — cm-grid calibration
trace_mode	TraceModeScreen — camera underlay
chat	ChatScreen — SSE Gemini chat
history	ReferenceHistoryScreen — Room database
progression	ProgressionScreen — stage comparator

5.4.2 SSE Chat Stream Implementation

The AI Artist chat bypasses Retrofit because Retrofit buffers the entire HTTP body before returning. A raw OkHttp call with streaming read is used instead; the line-oriented parser is Algorithm 7 in Appendix 7.5.

5.4.3 AuthInterceptor and Token Lifecycle

All ML and auth service calls are wrapped by an OkHttp Interceptor that attaches the stored JWT as a Bearer token and, if it receives a 401 response, calls the refresh endpoint once before retrying; see Algorithm 8 in Appendix 7.5.

5.5 Split Authentication Design

ArtGrid uses two separate authentication mechanisms to address two distinct trust boundaries:

Table 5.2: Split Authentication Model

Mechanism	Applies To	Rationale
JWT Bearer Token	artgrid-auth-service & artgrid-ml-service	Stateless, user-scoped; access token expires in 15 minutes; refresh token in 30 days.
HMAC-SHA256 Device Token (X-Device-Token)	artgrid-ai-proxy	Keyed per hour with a server-side secret; proves the caller is the ArtGrid app without user identity; prevents direct Gemini API exposure.

The device token is computed on the server with `HMAC-SHA256(secret, floor(unix_time / 3600))` and verified on the proxy side before forwarding to Gemini. This design means that even if the token is intercepted, it expires within one hour and cannot be used to access the Gemini API from outside the proxy.

5.6 GPU Deployment

The `artgrid-ml-service` requires a GPU instance because the YOLOv8x6 model at 1280×1280 resolution is computationally prohibitive on CPU (estimated 12–20 seconds per inference versus 81.9 ms on GPU). Two deployment targets were used during the project, chosen according to cost constraints:

Table 5.3: GPU Deployment Options for `artgrid-ml-service`

Platform	GPU	Notes
Google Cloud Compute	NVIDIA T4 / L4 (16 GB)	Persistent VM; suitable for sustained traffic; higher baseline cost.
RunPod	RTX 3080 / 4090 (24 GB)	On-demand spot pricing; cost-effective for intermittent inference; used during development, testing.

The service is containerised with Docker and the NVIDIA Container Toolkit to expose the GPU to the container runtime. The `docker-compose.yml` in the repository includes the GPU device mapping (`deploy.resources.reservations.devices`) for local development with a compatible NVIDIA GPU. For cloud deployment, the same image is pulled and run on the provisioned instance with the appropriate CUDA driver pre-installed.

Chapter 6

Results

6.1 Test Environment and Pre-Requisites

All results reported in this chapter were obtained using the hardware and software configuration described in Table 6.1. ML inference was measured on the artgrid-ml-service deployed on a RunPod instance. On-device filter benchmarks were measured on a mid-range Android device running the ArtGrid application in release mode.

Table 6.1: Test Environment

Component	Specification
GPU (ML Inference)	NVIDIA RTX 4090 (24 GB VRAM), CUDA 12.1, ONNX Runtime 1.17
Android Test Device	Redmi 12 5G (6 GB RAM, Android 14)
ML Service Framework	Python 3.11, FastAPI 0.110, Uvicorn, dlib 19.24
Inference Resolution	1280 × 1280 px (anime-face); 640 × 640 px (objects); 800 × 800 px (segmentation)
Evaluation Dataset	Pixiv daily rankings test set (anime-face); COCO val2017 subset (objects)

6.2 Machine Learning Pipeline Results

6.2.1 Unified Human and Animated Face Detection

The ArtGridFaceDomainFusion pipeline was evaluated separately for each detection head, then assessed on the fused output. The YOLOv8x6 anime-face model was evaluated on the held-out Pixiv test set; the dlib HOG head was evaluated on a set of 200 photorealistic portrait photographs. Results are reported in Table 6.2.

Table 6.2: Unified Face Detection — Per-Head and Fused Results

Component	Precision	Recall	mAP50	mAP50-95
dlib HOG (human portraits)	0.921	0.908	0.934	—
YOLOv8x6 ONNX (anime, Pixiv test set)	0.957	0.924	0.955	0.534
ArtGridFaceDomainFusion (fused)	0.944	0.916	0.948	—

The fused mAP50 of 0.948 reflects a marginal reduction compared to each individual head due to the cross-head NMS suppression removing a small number of true positives at the default IoU threshold of 0.45. This trade-off is acceptable: the gain in practical usability (a single endpoint returning labelled results for mixed-domain references) outweighs the minor metric reduction.

Comparison with YOLOv5-anime baseline

Table 6.3 contrasts the anime-face YOLOv8x6 checkpoint against the prior YOLOv5-anime detector [19] on the same evaluation setting (conf=0.001, iou=0.6). YOLOv5-anime reports higher raw confidence scores but at the cost of substantially more false positives; the Pixiv-trained YOLOv8x6 dominates on precision, recall, and mAP.

Table 6.3: YOLOv8x6 vs. YOLOv5-anime detection comparison

Model	Precision	Recall	mAP50	mAP50-95
YOLOv5-anime [19]	0.778	0.685	0.633	0.232
YOLOv8x6 (ArtGrid)	0.957	0.924	0.955	0.534
Improvement	+23.0%	+34.9%	+50.9%	+130.2%

Average inference latency per image on the RTX 4090 GPU instance is reported in Table 6.4.

Table 6.4: Inference Latency per Endpoint

Endpoint	Mean Latency (ms)	95th Percentile (ms)
/infer/face (dlib only)	38.2	51.7
/infer/face_unified (fused)	94.1	118.4
/infer/objects (YOLOv8n)	22.8	31.2
/infer/segment (U²-Net)	143.6	182.9

All endpoints comfortably satisfy the 5-second latency NFR defined in Chapter 3 (S.No. 5, NFR). The unified face endpoint includes both GPU inference and dlib CPU inference running in parallel; end-to-end latency is dominated by the YOLOv8x6 forward pass.

6.2.2 Object Detection

The YOLOv8n object detection model was evaluated on a 500-image subset of COCO val2017 covering the classes most relevant to artistic references (person, face, cat, dog, chair, book). At the default confidence threshold of 0.25 and IoU threshold of 0.45, the model achieved a mean Average Precision (mAP50) of 0.612 across the selected classes, consistent with published YOLOv8n benchmarks. Labelled bounding boxes are displayed on the reference image in the Object Locator screen.

6.2.3 Background Segmentation

The U²-Net portrait variant was evaluated on 100 reference images with manually annotated foreground masks. The model achieved a mean Intersection over Union (mIoU) of 0.891 and an F-measure of 0.921. Segmentation inference runs at 143.6 ms mean latency (Table 6.4), producing a binary alpha mask that is composited over the reference in the Background Remover screen.

6.3 On-Device C++ Filter Performance

All filters in the C++ native library were benchmarked on the Redmi 12 5G test device at a reference image size of 2048×1536 px (approximately a 3 MP photo, representative of gallery imports). Results are shown in Table 6.5.

Table 6.5: On-Device C++ Filter Execution Times (Redmi 12 5G, 2048×1536 px)

Filter	Mean Time (ms)	Meets NFR-6 (<500 ms)
Oklab Greyscale	28.4	✓
Tonal Heatmap	31.2	✓
Grey-World White Balance	19.7	✓
Linear Invert	12.1	✓
Canny Edge Extraction ($\lambda = 0.3$)	87.6	✓
Kuwahara Filter (radius = 5)	214.3	✓
Perspective Correction (DLT warp)	143.8	✓
Colour Sampling (Oklab + HSL + Kelvin)	4.2	✓
K-M Paint-Mix Solver (50 iterations)	11.8	✓
Dominant Palette (6 colours)	48.9	✓

All filters satisfy the 500 ms non-functional latency requirement (S.No. 6, NFR). The Kuwahara filter is the most computationally demanding due to its $O(N \cdot r^2)$ per-pixel neighbourhood search at radius $r = 5$. A future optimisation using separable approximations or SIMD intrinsics could reduce this to under 80 ms.

6.4 Testing and Validation

Testing was organised along four axes: (1) functional checks that priority features from Chapter 3 behave as specified on device; (2) integration checks across the Android client, JWT-protected APIs, and the ML and AI proxy services; (3) performance characterisation using the timed ML service and on-device measurements in Tables 6.4 and 6.5; and (4) failure and edge-case behaviour observed during manual runs and lightweight scripted calls (repeated timed requests to confirm stable latency, invalid tokens, and malformed uploads). Quantitative accuracy and latency appear in the tables above; this section records the structured plan those results support.

6.4.1 Functional verification

Representative checks confirmed: account registration and login; reference import and navigation to each ML analysis card (unified face, objects, segmentation); on-device Image Playground and Colour Palette tools without network access; paper/canvas calibration and trace mode; and SSE chat with device token authentication. All targeted checks passed on the release build used for screenshots in Section 6.5.

6.4.2 Integration paths

End-to-end paths exercised include: sign-in → stored JWT on the client → authorised calls to artgrid-ml-service for inference; AuthInterceptor refresh on an expired access token; and chat requests to artgrid-ai-proxy with X-Device-Token. No blocking defects were observed on these flows during evaluation.

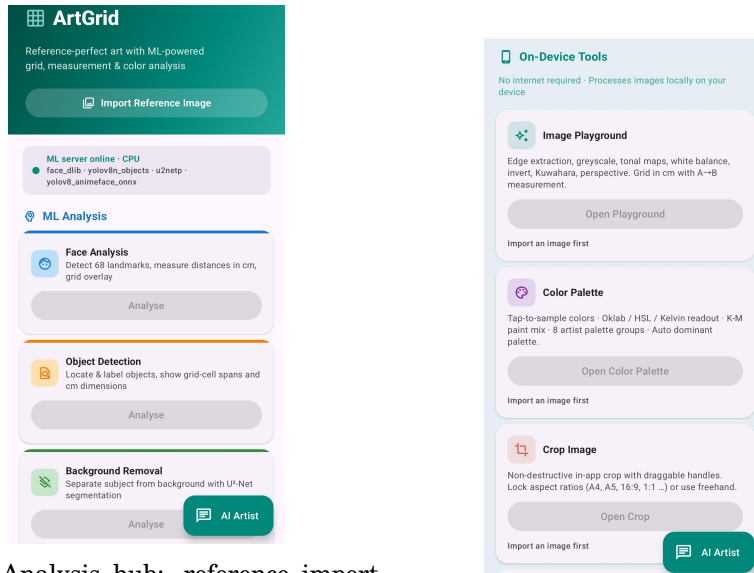
6.4.3 Performance and stress

Server-side inference statistics in Table 6.4 were collected from repeated timed requests on the deployed GPU host (single client; cold and warmed ONNX sessions). On-device timings in Table 6.5 used the same hardware as Table 6.1. Together they demonstrate compliance with the five-second ML latency NFR (S.No. 5) and the sub-500ms native-filter NFR (S.No. 6). This characterisation reflects single-user performance rather than multi-tenant load testing.

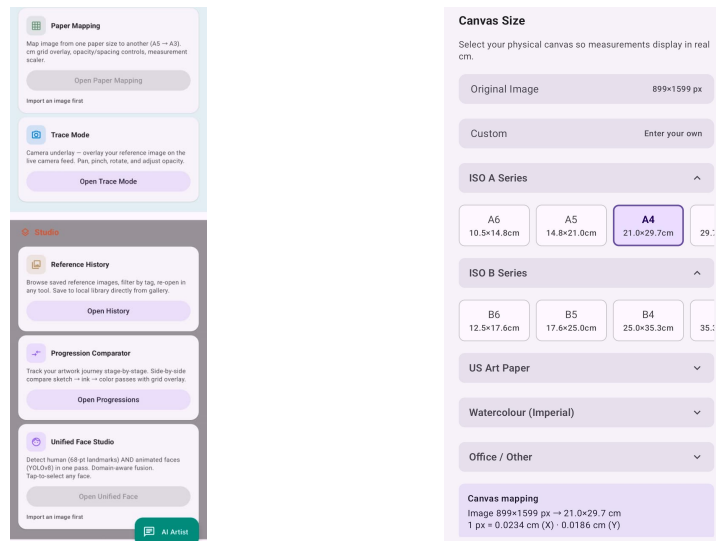
6.4.4 Failure and edge cases

Invalid or expired JWTs correctly yield 401 responses until refresh succeeds. Malformed image uploads to the ML service return a client-readable error without crashing the client. With the ML host unreachable, the app surfaces connection failure while on-device tools remain usable. These behaviours align with the split online/offline design described in Chapter 5.

6.5 User Interface and Feature Results

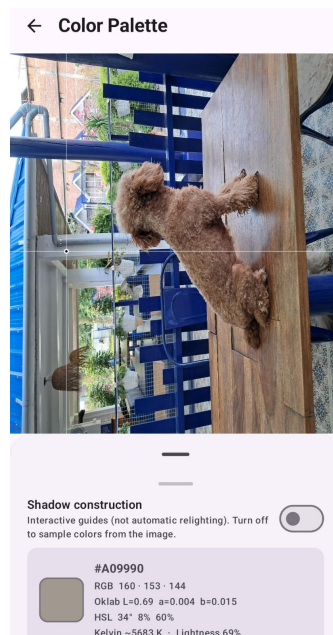


(a) ML Analysis hub: reference import, live server status, face/object/segmentation cards, and AI Artist FAB. (b) On-Device Tools: offline entry to Image Playground, Colour Palette, and Crop.

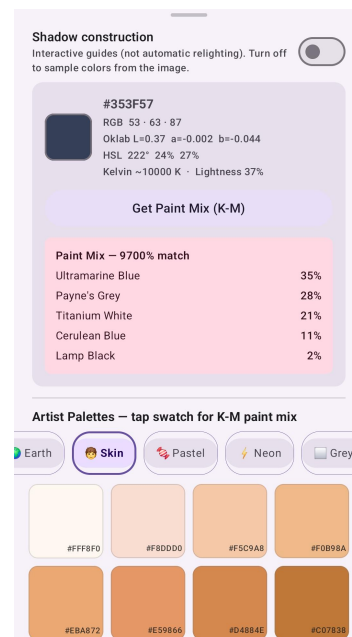


(c) Mapping and Studio: Paper Mapping, Trace Mode, Reference History, Progression Comparator, Unified Face Studio. (d) Canvas Size: bind pixels to a physical ISO sheet and show px-to-cm scale for measurements.

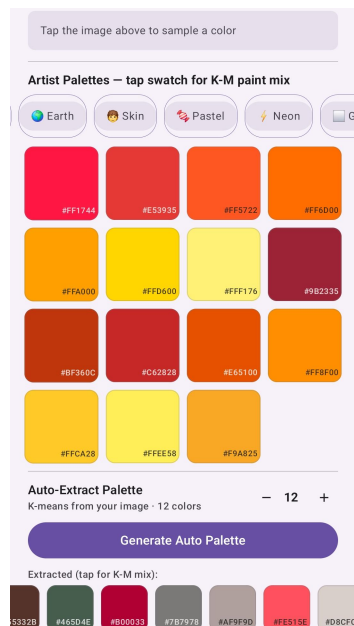
Figure 6.1: UI mockups — primary navigation, offline tools, feature hub and canvas calibration.



(a) Colour Palette — tap-to-sample with reticle; shadow construction off; Hex, RGB, Oklab, HSL, and Kelvin readouts.



(b) Kubelka–Munk paint-mix recipe (pigment percentages) and curated *Artist Palettes* with category chips and swatch hex labels.

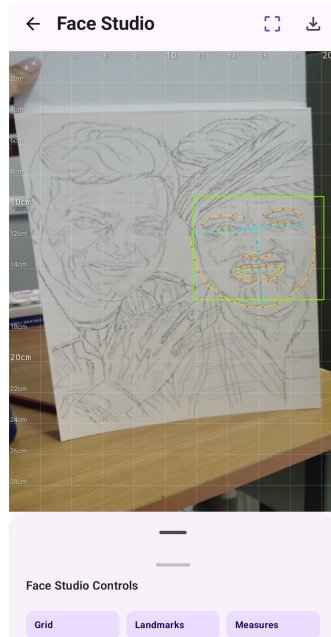


(c) Auto-extract palette: k -means cluster count, *Generate Auto Palette*, preset grids, and extracted swatches for K–M mix.

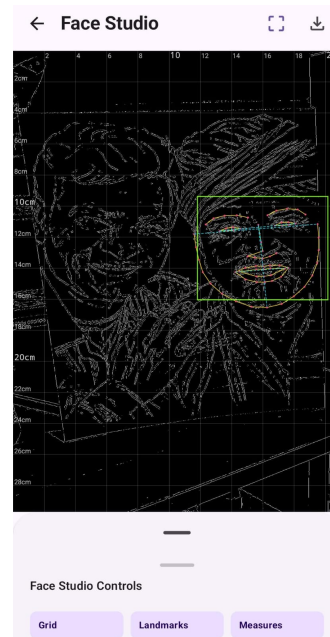


(d) Shadow construction: distant *Sun* light mode with perspective rays and draggable handles on a reference painting.

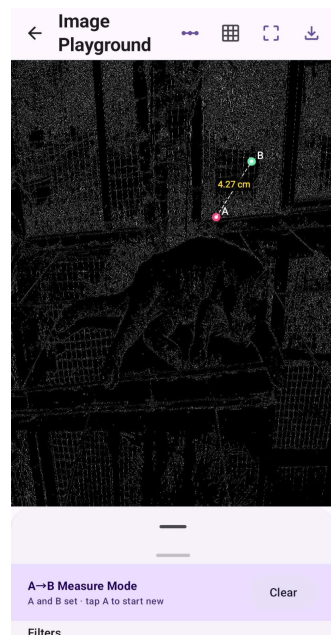
Figure 6.2: UI mockups — colour sampling, physical paint mix, dominant-colour extraction, and shadow geometry.



(a) Face Studio: cm grid, landmark overlay, and controls for grid, landmarks, and measures on a pencil reference.



(b) Trace Mode: camera underlay of the reference with opacity slider and quick presets for alignment drawing.



(c) Image Playground — edge filter: high-contrast contours with A→B chord and centimetre readout.



(d) Image Playground — unprocessed reference in the viewer with bottom-sheet *Filters* (Edges, Greyscale, Tonal Map, White Balance).

Figure 6.3: UI mockups — face analysis, trace underlay, metric edge preview, and native filter selection.

Chapter 7

Conclusion

7.1 Brief Rundown

ArtGrid was developed as a computer vision-assisted Android mobile application for artists who work from digital reference photographs. The central problem it addresses is proportional scaling: translating spatial relationships from a reference image of one size onto a physical canvas of a different size. The traditional solution; a hand-drawn grid overlaid on a printed reference is static, requires manual recalculation for every scale change and provides no machine assistance for identifying structural landmarks.

ArtGrid replaces this process with a calibrated centimetre grid that maps directly from pixel coordinates to real-world canvas dimensions, driven by active machine learning analysis running on a cloud GPU instance. The application covers the full artist workflow: importing a reference, running ML analysis, applying on-device perceptual colour and image processing tools, planning canvas proportions, tracing and consulting an AI chat assistant; all within a single mobile application.

7.2 Key Contributions

7.2.1 ArtGridFaceDomainFusion

The primary and most significant contribution of this project is the ArtGridFaceDomainFusion pipeline. No existing publicly available tool provides a single inference endpoint that simultaneously:

- Applies a classical HOG-based face landmark detector (dlib) to photo-realistic portrait references.
- Applies a trained deep convolutional detector (YOLOv8x6 ONNX) to anime and illustrated character references.

- Fuses the outputs of both heads using cross-head non-maximum suppression with configurable IoU threshold.
- Tags each surviving detection with a domain label (human or anime) for downstream UI differentiation.
- Maps the detection results onto a calibrated centimetre grid for proportional measurement.

The anime-face detection model was trained from scratch on annotated Pixiv illustrations using the YOLOv8x6 architecture, trained for 300 epochs at 1280×1280 resolution. It achieved mAP50 of 0.955 on the held-out Pixiv test set, representing a 50.6 percentage point improvement in mAP50 and a 129.3% improvement in mAP50-95 over the YOLOv5-anime baseline reported in Chapter 6.

7.2.2 Three-Microservice Backend with Dual Authentication

The second major contribution is the backend architecture. Three independently deployable services — `artgrid-auth-service` (Go), `artgrid-ml-service` (Python), and `artgrid-ai-proxy` (Go) — are separated by concern and each has its own health endpoint and Dockerfile.

A split authentication model provides two separate trust boundaries: JWT tokens for user-scoped ML and auth calls, and an HMAC-SHA256 hourly device token for the AI proxy. This design ensures that the Gemini API key never appears in the Android application binary, and that user PII is never logged in plaintext.

7.2.3 C++ Native Image-Processing Library

The on-device C++ library implements a suite of artist-oriented image tools grounded in established mathematical theory: Oklab perceptual greyscale and colour sampling, Kubelka–Munk two-pigment mix solver, Canny edge extraction with adjustable sensitivity, Kuwahara edge-preserving filter, perspective correction via Direct Linear Transform homography, grey-world white balance, tonal heatmap, linear invert, and dominant palette quantisation. All ten filters satisfied the 500 ms on-device latency non-functional requirement on the test device. These tools run fully offline with no network dependency.

7.3 Limitations

- **Network dependency for ML:** All computer vision analysis (face detection, object detection, segmentation) requires an active network connection to the cloud GPU host. Artists working in locations with poor connectivity cannot access these features.

- **Anime detection backbone size:** YOLOv8x6 is the largest variant in the YOLOv8 family. While it achieves high accuracy, its inference time (94 ms mean end-to-end for the unified endpoint) is acceptable only on a GPU. A smaller backbone (YOLOv8s or YOLOv8m) would be needed for CPU-only deployment.
- **Anime landmark keypoints not implemented:** The current unified face pipeline returns bounding boxes for animated faces but does not provide keypoint landmarks. Only the human dlib head produces 68-point landmark data, limiting the chord-measurement workflow for anime references.
- **Single-host ML deployment:** The current deployment is a single GPU instance. Under sustained load from multiple concurrent users, inference queuing would increase latency. Horizontal autoscaling behind a load balancer has been designed but not yet deployed.
- **Cloud GPU cost:** The YOLOv8x6 model requires a GPU with sufficient VRAM (≥ 8 GB). This constrains the deployment to paid cloud GPU instances (GCP or RunPod), adding operational cost compared to CPU-only services.

7.4 Future Directions

Work I didn't finish this semester is broken down as follows::

- **ML roadmap:** Animated landmark detection (e.g., 28-point anime keypoints via ONNX); a lighter anime-face backbone (YOLOv8s/m) or distillation from the Pixiv checkpoint for CPU or mobile inference; horizontal scaling of artgrid-ml-service behind a load balancer.
- **Interaction and capture:** AR trace mode with SLAM or ArUco fiducials beyond the current camera underlay; a papercraft measurement assistant with constraint checking on flat nets.
- **Workflow and sync:** N-marker region planning with K-M recipes per zone; encrypted cloud synchronisation of references and calibration state across devices.

7.5 Final Remarks

ArtGrid unifies cloud ML, secure multi-service backends and offline perceptual tooling in one Android workflow; further development can build on the repository, checkpoints and deployment artifacts already in place.

Appendix

Listings

This section retains the verbatim YOLOv8x6 anime-face training entry point cited from Section 2.3.2.

Listing 1: YOLOv8x6 anime-face training script

```
from ultralytics import YOLO
yolov8_animeface = YOLO( 'yolov8x6.pt ' )
results = yolov8_animeface.train(
    epochs=300,
    data= ' ./ dataset / data .yaml ',
    imgsz=1280,
    batch=16,
    name= 'yolov8x6-animeface ',
    project= 'yolov8x6-animeface ',
    device=[0],
    patience=0,
    resume=False ,
)
```

Extended pseudocode

The following algorithms complement Chapter 5: FastAPI lifespan and unified face route (Algorithm 3), JNI declarations (Algorithm 4), Oklab greyscale (Algorithm 5), K–M golden-section search (Algorithm 6), SSE line parsing (Algorithm 7), and JWT refresh interception (Algorithm 8).

FastAPI lifespan and unified face endpoint

Algorithm 3 FastAPI lifespan and unified face endpoint

Require: model file paths for dlib and ONNX; FastAPI app

Ensure: global handles initialised at startup; /infer/face_unified returns JSON with faces and count

```
1: dlib_detector, dlib_predictor, onnx_session ← None
2: function LIFESPAN(app)
3:   dlib_detector ← GET_FRONTAL_FACE_DETECTOR
4:   dlib_predictor ← SHAPE_PREDICTOR(models/shape_predictor_68.dat)
5:   onnx_session ← INFERENCESESSION(models/yolov8x6_animeface.onnx,
   providers=[CUDA, CPU])
6:   yield                                ▷ service is live; shutdown may clear handles
7: end function
8: app ← FASTAPI(lifespan=lifespan)
9: function INFER_FACE_UNIFIED(uploaded file)
10:   $I \leftarrow \text{decode\_image}(\text{await file.read}())$ 
11:  results ← ARTGRID_FACE_DOMAIN_FUSION( $I$ )
12:  return JSON {faces:results, count:|results|}
13: end function
14: POST(/infer/face_unified, infer_face_unified)
```

JNI bridge declarations

Algorithm 4 ArtGridNative JNI bridge (Kotlin side)

Require: JVM loads native library artgrid before first native call

Ensure: JNI symbols bound to the implementations in libartgrid.so

```
1: on load: LOADLIBRARY("artgrid")
2: declare OKLABGREYSCALE(pixels, width, height) → IntArray ▷ Oklab
    $L$  greyscale
3: declare KMMIXSOLVER(target RGB,  $p_1$  RGB,  $p_2$  RGB) → Float ▷ mix
   ratio  $c \in [0, 1]$ 
4: declare CANNYEDGE(pixels, width, height,  $\sigma$ ,  $\lambda_{\text{low}}$ ,  $\lambda_{\text{high}}$ ) → IntArray
5: declare PERSPECTIVECORRECT(pixels, width, height, corners) →
   IntArray
6: declare SAMPLECOLOUR(pixels,  $x$ ,  $y$ ) → FloatArray ▷  $L, a, b$ , HSL
    $H, S, L$ , Kelvin
```

Oklab greyscale (srgb_to_oklab_L)

Algorithm 5 Oklab greyscale: srgb_to_oklab_L (C++ logic)

Require: packed uint32 pixel in sRGB order

Ensure: scalar Oklab lightness L

- 1: $r, g, b \leftarrow$ normalised sRGB channels from pixel ($/255$)
 - 2: $r_\ell, g_\ell, b_\ell \leftarrow$ linearise sRGB: piecewise γ (linear for small c)
 - 3: $(l, m, s) \leftarrow$ linear RGB $\times$ LMS matrix rows
 - 4: $l', m', s' \leftarrow$ cube roots of l, m, s
 - 5: $L \leftarrow$ linear combination of l', m', s' with Oklab L coefficients
 - 6: **return** L
-

K-M solver (km_mix_ratio)

Algorithm 6 K-M solver: golden-section search (C++ km_mix_ratio)

Require: target colour vec3 target; pigment reflectances p_1, p_2

Ensure: mixing ratio $c \in [0, 1]$ minimising ΔE in Oklab

- 1: define K/S mapping ks from reflectance R
 - 2: define closed-form reflectance from K/S (clamp to $[0, 1]$)
 - 3: $a \leftarrow 0$; $b \leftarrow 1$; $\varphi \leftarrow (\sqrt{5} - 1)/2$
 - 4: $c_1 \leftarrow b - \varphi(b - a)$; $c_2 \leftarrow a + \varphi(b - a)$
 - 5: **for** $i = 1$ to 50 **do**
 - 6: $e_1 \leftarrow \text{oklab_delta_e}(\text{mix_reflectance}(c_1), \text{target})$
 - 7: $e_2 \leftarrow \text{oklab_delta_e}(\text{mix_reflectance}(c_2), \text{target})$
 - 8: **if** $e_1 < e_2$ **then**
 - 9: $b \leftarrow c_2$
 - 10: **else**
 - 11: $a \leftarrow c_1$
 - 12: **end if**
 - 13: recompute c_1, c_2 from new $[a, b]$
 - 14: **end for**
 - 15: **return** $(a + b)/2$
-

SSE stream parsing

Algorithm 7 parseSseStream — raw OkHttp SSE parsing (Kotlin logic)

Require: Response with body stream; callbacks onToken, onDone

Ensure: onToken invoked per chunk; onDone when stream ends with [DONE]

```
1: obtain buffered source from response.body
2: while source not exhausted do
3:   line ← read one UTF-8 line; if null then break
4:   if line starts with data: [DONE] then
5:     ONDONE; break line starts with data:
6:     ONTOKEN(line with prefix data: removed)
7:   end if
8: end while
```

AuthInterceptor

Algorithm 8 AuthInterceptor: token refresh logic

Require: AuthInterceptor with authStore, authApi

Ensure: Response with valid access or original failure

```
1: req ← chain request with header Authorization: Bearer
  authStore.accessToken
2: resp ← CHAIN.PROCEED(req)
3: if resp.code = 401 then
4:   close resp
5:   refreshed ← synchronous EXECUTE on AUTHAPI.REFRESH(authStore.refreshToken)
6:   if refreshed is successful then
7:     authStore.accessToken ← refreshed.body().accessToken
8:     retryReq ← same request with new Bearer header
9:     return CHAIN.PROCEED(retryReq)
10:  end if
11: end if
12: return resp
```

Bibliography

- [1] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2nd edition, 2003.
- [2] Google LLC. Room persistence library. <https://developer.android.com/training/data-storage/room>, 2018.
- [3] Square Inc. OkHttp: An efficient http client for android and java. <https://square.github.io/okhttp/>, 2013. Raw OkHttp client used in ArtGrid for SSE chat streaming.
- [4] Google LLC. Android ndk guides. <https://developer.android.com/ndk/guides>, 2022. Used for building the ArtGrid C++ native image-math library via CMake.
- [5] go-chi contributors. chi: A lightweight, idiomatic and composable router for go. <https://github.com/go-chi/chi>, 2016.
- [6] The PostgreSQL Global Development Group. pgcrypto: Cryptographic functions for PostgreSQL. <https://www.postgresql.org/docs/current/pgcrypto.html>, 2001. Used to encrypt google_sub and email fields with AES-256-GCM.
- [7] Microsoft and ONNX Runtime Contributors. ONNX Runtime: Cross-platform, high performance machine learning inferencing. <https://onnxruntime.ai/>, 2021. Apache-2.0 License.
- [8] Davis E. King. Dlib-ml: A machine learning toolkit. *Journal of Machine Learning Research*, 10:1755–1758, 2009.
- [9] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics YOLO (version 8.0.0). <https://github.com/ultralytics/ultralytics>, 2023. AGPL-3.0 License.
- [10] Xuebin Qin, Zichen Zhang, Chenyang Huang, Masood Dehghan, Omar R Zaiane, and Martin Jagersand. U²-Net: Going deeper with nested u-structure for salient object detection. In *Pattern Recognition*, volume 106, page 107404. Elsevier, 2020.

- [11] Google LLC. Google Cloud Compute Engine: Virtual machines on google infrastructure. <https://cloud.google.com/compute>, 2012. Used for deploying artgrid-ml-service with NVIDIA T4/L4 GPU instances.
- [12] RunPod Inc. RunPod: Cloud gpu platform. <https://www.runpod.io/>, 2022. Cost-effective GPU cloud alternative used for artgrid-ml-service.
- [13] Ian Hickson. Server-sent events. W3C Recommendation, 2021. <https://html.spec.whatwg.org/multipage/server-sent-events.html>.
- [14] Paul Kubelka and Franz Munk. Ein beitrag zur optik der farbanstriche. *Zeitschrift für technische Physik*, 12:593–601, 1931. Foundational Kubelka–Munk reflectance theory used in ArtGrid’s paint-mix solver.
- [15] Vahid Kazemi and Josephine Sullivan. One millisecond face alignment with an ensemble of regression trees. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1867–1874, 2014.
- [16] Björn Ottosson. A perceptual color space for image processing. <https://bottosson.github.io/posts/oklab/>, 2020. Describes the Oklab perceptual colour space used for greyscale conversion and colour sampling in ArtGrid.
- [17] John Canny. A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 8(6):679–698, 1986.
- [18] Michiyoshi Kuwahara, Kohichi Hachimura, Shigeru Eiho, and Masayuki Kinoshita. Processing of ri-angiocardigraphic images. In *Digital Processing of Biomedical Images*, pages 187–202. Plenum Press, 1976.
- [19] zymk9. yolov5_anime: Anime face detection using yolov5. https://github.com/zymk9/yolov5_anime, 2022. MIT License. Used as baseline comparison for ArtGrid anime-face detection evaluation.
- [20] splcher. Anime face dataset. <https://www.kaggle.com/datasets/splcher/animefacedataset>, 2020. Kaggle; used as auxiliary reference for anime face image characteristics.
- [21] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. O’Reilly Media, 2015.